Court Vision: Motion Planning for an Obstacle-Aware Ball-Dribbling Mobile Robot

RBE 550 Motion Planning — Final Report

Rudy Arsenault, Tracy Yang
Worcester Polytechnic Institute

Abstract—We present a simulated mobile robot that dribbles a ball with a paddle end-effector through a cluttered course, choosing where each bounce lands so that the ball, the arm, and the chassis all stay clear of obstacles. An offline planner searches over position, heading, and *time*, then over the arm angle along the result, proving a run collision-free in advance when obstacle motion is known; a schedule-tracking controller keeps the real run within 6.3 cm of that plan (0.6 cm on average). Letting the planner choose where the ball lands, rather than fixing it beside the chassis, solves 25 of 60 dense courses against 7. A live planner handles obstacles that roam at their own speeds, turn at random, and bounce off each other and the robot: it senses them at every contact, predicts their bounces, routes with D* Lite, and picks each step with a dynamic-window local planner, finishing 27 of 30 roaming courses against 19 for D* Lite alone and 9 for hybrid A* replanned online. The robot slows in narrow passages, speeds up in the open and when chased, and keeps the ball in bounds. Paddle contacts are rigid-body impacts whose energy is fully accounted for.

TO DO: Update the abstract with the Objective 5 evaluation and terrain results.

I. Introduction

A basketball player dribbling through a crowd solves two coupled problems at once: where to send the ball next, and where to put their body so they can meet it there. Our robot faces the same coupling. Its base cannot simply steer around clutter, because it must be at the ball's next contact point; when the nominal next bounce is blocked, the robot must instead *redirect* the ball to a reachable, collision-free spot and plan the base to meet it. The proposal set five objectives: (1) a dribble model, (2) a mobile base that dribbles while moving, (3) a static obstacle course, (4) a redirect-the-bounce planner, and (5) an evaluation, with a stretch goal of obstacles that move during a run.

Objectives 1–3 were complete at the Week 8 status update. This report covers the redirect planner and its test against a no-redirect baseline (Objective 4), its extension to obstacles moving on a known schedule and then to obstacles whose motion must be sensed and predicted (the stretch goal), an energy-consistent contact model, mobile-base limits, the success criteria we hold the system to, and the evaluation to date (Objective 5).

II. Background and Related Work

Dribbling. Bühler, Koditschek, and Kindlmann formalized juggling as an intermittent dynamical task and introduced the mirror law, a feedback rule whose end-effector motion is a distorted reflection of the ball's [1]. Bätz et al. showed basketball dribbling shares the same hybrid impact/flight dynamics and demonstrated it on a stationary arm [2, 3]. DribbleBot moved dribbling onto a walking quadruped over rough terrain, without discrete obstacles or an explicit planning layer [4].

Planning among obstacles. Hybrid A* searches a lattice of short kinematically feasible arcs over continuous position and heading [5]; we extend its state with time. D* Lite incrementally repairs a shortest-path solution as edge costs change and the robot moves, re-expanding only what the change invalidates [6]. The dynamic window approach selects velocities by forward-simulating candidates and scoring progress against clearance [7], and velocity obstacles reason about collisions with moving obstacles in velocity space [8]. Our live planner pairs a D* Lite global route with a dynamic-window local step, a standard global/local split.

III. Success Criteria

Prompted by feedback on the status update, we fixed in advance what counts as success, so results are judged against targets rather than described after the fact. A live run *finishes* only if the robot crosses the finish line still dribbling; it otherwise ends the moment the dribble does, as *dribble lost* (the ball escaped the paddle), *out of bounds*, or *timeout* (120 s). Table 1 lists each criterion and where it stands; Section VII has the evidence.

IV. System Overview

Everything runs in PyBullet at 240 Hz. The ball is a dynamic sphere; the chassis, arm, and paddle are kinematic bodies posed every step (Fig. 1). The paddle sits over the ball's real position and the chassis one arm length (0.5 m) away at the arm angle ϕ, which is also the chassis's yaw: chassis and arm turn together as one rigid assembly. Each contact, the planner chooses where the ball lands next and ϕ.

Mobile-base limits. The chassis may move at most 0.5 m/s and turn at most 90 °/s; because its yaw is ϕ, the turn-rate limit also caps how fast the arm angle can change. The ball's horizontal speed is capped at 0.7 m/s, leaving the arm room to move the ball relative to a chassis already at cruise. Both planners enforce all three.

Speed that follows the space. The robot's cruising speed depends on how much room it has: 0.15 m/s with no free space between the chassis and the nearest static obstacle, rising linearly to 0.45 m/s at 0.5 m or more. Only static obsta-

Criterion	Result
Offline: zero collisions on every course it accepts	Met: 0 in 31 runs
Offline: ball within 5 cm of its plan	Not met: 6.3 cm worst, 0.6 cm mean
Redirect solves more courses than no-redirect	Met: 25 vs. 7 of 60
Live: $\geq$90% of roaming courses finished	Met on the 30 shared courses (27); 77% (46/60) on a wider set
Live: $\leq$1 collision per run on average	Met: 0.43 (0.58 on 60)
Live: mean replan within one dribble cycle ($\approx$300 ms)	Met: 119 ms (worst 492 ms)
Unexplained ball energy $<$1% of paddle work	Met: 0.06% mean
Paddle and base within actuator limits	Met by construction
Completion degrades gracefully with density	*Pending (Obj. 5)*

Table 1: Success criteria and status.

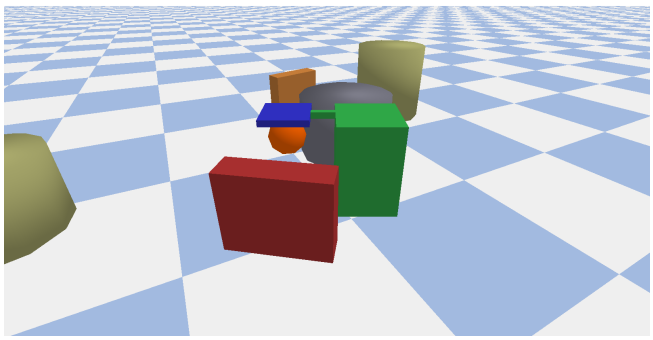


Figure 1: The robot threading a gap: green chassis, arm, blue paddle over the orange ball. Tinted obstacles are moving.

cles count. Narrow passages are fixed geometry, and in our first version, slowing down near moving obstacles only gave them longer to reach the robot (24 vs. 26 of 30 finishes).

Courses (Objective 3). Each course draws crates, pillars, and walls of random size, position, and orientation for a 6×3 m lane (Fig. 2), keeping each only if it is at least 0.4 m from every other obstacle's bounding circle. This gives 6.2 obstacles per roaming course on average and 4.3 per oscillating course, since an oscillating obstacle also reserves its whole sweep. About half the obstacles move, in one of two ways:

- *Oscillating* across the lane on a known sinusoid (0.3–0.6 m amplitude, 4–8 s period).
- *Roaming*: each obstacle has its own base speed, drawn from 0.05–0.40 m/s, so some crawl and some outrun the robot's cruise. It turns at random moments (on average every 2.5 s) by up to $\pm 120°$, varying its speed by up to $\pm 30\%$ of its base. It bounces off the course edges, the static obstacles, each other, and the robot's chassis, using the chassis's true shape.

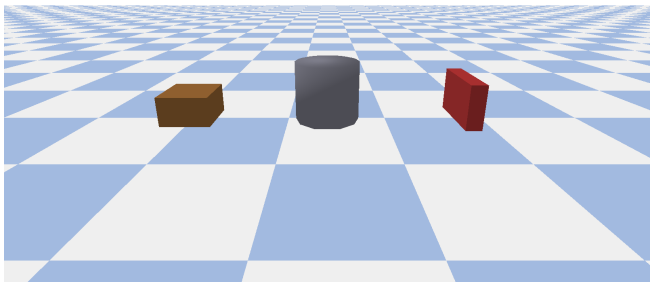


Figure 2: The three obstacle kinds and their size ranges.

Collisions between obstacles are detected before each step, so they never overlap: over 100 roaming courses simulated for 300 s (303 movers), the closest two came was touching (0.002 mm apart). Turns are drawn from the course seed, so runs are reproducible, but no planner sees the turn schedule.

V. Dribble Model and Energy-Consistent Contact (Objective 1)

Between contacts the ball is a free projectile, bouncing off the floor with PyBullet's restitution model (coefficient 0.85). We removed PyBullet's default linear damping (0.04 s^{-1}), an unmodeled drag that the planners' ballistic timing model does not include.

At each paddle contact, the vertical outgoing velocity follows the mirror law: $-v_{\text{push}} = -2.2$ m/s whatever the incoming speed, so the bounce height stays constant [1]. This gives 3.3 contacts per second; we slowed it from 3.0 m/s (4.3 per second) for a more deliberate dribble, and much below 2 m/s the ball would barely return to the paddle. The horizontal outgoing velocity $\mathbf{v}_{xy}$ is the planner's push. Earlier versions simply overwrote the ball's velocity, with no impulse, no paddle work, and no account of where the added energy came from. The contact is now a frictionless rigid-body impact between the ball and a much heavier paddle. With incoming velocity $\mathbf{v}$, desired outgoing velocity $\mathbf{v}'$, and $\Delta\mathbf{v} = \mathbf{v}' - \mathbf{v}$, the impulse must lie along the face normal, so the paddle face tilts to $\mathbf{n} = \Delta\mathbf{v}/\|\Delta\mathbf{v}\|$ and is driven at normal speed

$$u = \frac{\|\Delta\mathbf{v}\|}{1+e} + \mathbf{v}\cdot\mathbf{n}, \tag{1}$$

with paddle restitution $e = 0.8$. The paddle does work $W = m\|\Delta\mathbf{v}\|\,u$, of which $\frac{1}{2}m(1-e^2)(u-\mathbf{v}\cdot\mathbf{n})^2$ is lost in the impact and the rest becomes the ball's kinetic energy. Tilting the face is literally the proposal's "alternate hit angle" redirect; a 30 ° tilt limit and 3 m/s paddle-speed limit bound the actuator.

Every run keeps an energy ledger: each step's change in the ball's mechanical energy (translational, spin, and potential) is attributed to paddle work and impact loss at contacts, floor loss while touching the floor, and the integrator's known loss of $\frac{1}{2}mg^2\Delta t^2$ per step. What is left over is unexplained.

TO DO: Optional: figure of the energy ledger over one run (paddle work vs. floor and impact losses).

VI. Planning

A. Offline: Space–Time Hybrid A* with Known Obstacle Motion

When every obstacle's future position is known, the whole run can be planned and proven collision-free before it starts. The robot body's center is searched over a lattice of (x, y, θ, t): each edge is a 0.1 m forward arc at one of five turning rates, taking as long as the cruising speed at that point allows, so the plan slows through narrow gaps. Collision checks along each edge use each obstacle's position at the interpolated time. A *wait* edge holds the pose while dribbling in place, offered only near moving obstacles and capped at 4 s in total. Each search is capped at 200,000 node expansions so a hopeless course fails fast and is regenerated.

A second Dijkstra search over the found path picks ϕ at every point from 23 candidates, verifying every *transition* between consecutive points with position, angle, and time interpolated together. Transitions must respect the base speed, turn-rate, and ball speed limits, and each moving obstacle is inflated by its peak speed times ± 0.25 s, so a small timing error cannot become a collision. Landing spots must stay 5 cm inside the side lines and no more than 0.25 m behind the start line.

A plan proven at specific times holds only if the robot keeps to those times. At each contact the controller aims the ball at where the plan puts the paddle at the *next* contact's time, pushing at whatever speed covers that distance, and reads ϕ off the same clock.

B. Redirect-the-Bounce and a No-Redirect Baseline (Objective 4)

Because the arm-angle search chooses every landing point, redirection happens by construction. To test whether it matters, we built the baseline suggested in feedback on the status update: the same planner, but with the ball rigidly fixed beside the chassis ($\phi = \theta \pm 90°$, side chosen once per course). The ball's fixed position becomes part of the robot's footprint in the body search, which also rejects any turn the attached ball could not follow within the speed limits.

C. Live: D* Lite with a Dynamic-Window Local Step

Roaming obstacles turn at random and bounce off each other and the robot, so their future is not known in advance. The live planner may only read where each obstacle is *now*, through a sensor that estimates velocity from two readings 1/60 s apart (a 60 Hz camera), and replans at every contact.

Prediction. At each contact, the planner rolls every obstacle forward 2.5 s from its sensed position and velocity under the obstacles' own motion rule: straight lines, bouncing off the edges, the static obstacles, and each other. This assumes the robot knows how obstacles bounce, but not when they will turn. Without turns, it predicts 2 s ahead to within 0.3 cm on average, where assuming constant velocity is off by 12 cm (54 cm in bad cases); with random turns, the 95th-percentile error 2 s ahead is 41 cm.

Global route. The lane is a 70×31 grid of 0.1 m cells. D* Lite [6] searches backward from every finish-line cell over cells the robot body can occupy given the static obstacles. Moving obstacles enter only as cost penalties along their predicted sweeps, fading over 2.5 s, so a passable course always has a route. When penalties change between readings, only the affected vertices are updated. Against plain Dijkstra on 8,000 random replans, D* Lite returned an optimal path every time.

Local step. A dynamic-window step [7] evaluates 33 body velocities (standing still, plus 16 directions at the cruising speed and at an evasive 0.5 m/s) by rolling each forward 2 s against the predicted obstacles. The cruising speed comes from the free space along the next 0.6 m of route. Velocities that reach a static obstacle within 0.4 s, or a moving one within 1.0 s, are discarded, as are moves that would leave no in-bounds landing spot for the ball. The rest are scored by progress toward a point 0.3 m along the route (capped at the cruising speed), clearance up to 0.5 m, time to collision, and a cost for the evasive speed. When an obstacle within 1.5 m is behind the robot, closing, and would reach it within 2 s, the robot is *chased* and cruises at the evasive speed instead.

Arm angle. ϕ is chosen per contact from the same 23 candidates: within the turn-rate limit, with the landing point reachable within the ball speed limit, 5 cm inside the side lines, and clear of every obstacle's predicted position along the ball's flight and for 1 s after. Of those, the one closest to the current angle wins, with a mild preference for the ball beside the chassis and a penalty for landing within 0.25 m of a side line, so the arm starts steering the ball inward before it can be trapped against the line. When no candidate is fully clear, the fallback is the in-bounds spot with the most room from obstacles and the side lines.

D. Logging What Binds

To answer whether obstacles or the base's speed limit the robot, both planners record what constrained each decision. Offline: at each path point, whether both nominal landing spots beside the chassis were blocked by obstacles; at each transition, whether the turn-rate, base-speed, or ball-speed limit was at least 95% used. Live: at each contact, which constraint ruled out the arm angle the robot would otherwise have preferred, and whether the local step gave up progress to keep clear.

VII. Results

All runs are headless and deterministic for a given seed. Collisions are ground truth: the real chassis and ball against every obstacle's real position at the real time, with no planning margin.

A. Offline Planner

On the 10-course benchmark (`planner_benchmark.py`, oscillating obstacles), the planner needed 3.0 course generations on average (worst 8), took 15.3 s (worst 49.8 s), and produced paths 1.7–23.3% longer than the straight line (mean

7.3%); 7–34% of path points (mean 20%) were redirected away from the straight-ahead landing spot. Executing 20 offline plans with the schedule-tracking controller, all 20 finished with **zero overlaps** with any obstacle, and the ball stayed within 0.6 cm of its planned position on average. The worst deviation was 6.3 cm, missing our 5 cm target; it was 3.2 cm before the plan's speed varied with clearance, and the sharper speed changes between steps are the likely cause. The 99th percentile stayed below 4.5 cm on every course. Before the controller tracked the plan's clock, runs fell 3–19 s behind schedule and 4 of 5 test courses collided with moving obstacles.

B. Does Redirecting the Ball Matter?

Table 2 plans 60 raw courses (4.3 obstacles, 1.5 moving on average) with and without redirection, and records at which stage each failure occurred.

	Solved	No body path	No arm seq.
With redirect	25	24	11
No redirect	7	53	0

Table 2: Objective 4 ablation on 60 raw courses. "No arm seq.": a robot body path exists but no arm-angle sequence along it.

Redirection more than triples the courses solved: 18 are solved only with it and none only without, and where both succeed the paths are the same length (6.31 vs. 6.30 m). Most failures are the body search finding no route at all. But 11 of the full planner's failures found a body path with no workable arm sequence along it: the full planner's body search ignores the arm, so it can commit to a route the arm cannot follow.

C. What Binds: Obstacles or the Base?

Across the 11 oracle plans on roaming courses, both nominal landing spots were blocked by obstacles at 27% of path points, while the base turn-rate and speed limits were never binding and the ball speed limit bound on 0.4% of transitions. Live, obstacles ruled out the preferred arm angle at 10% of contacts and the side lines at 4%; the ball-speed limit did so at 3% and the turn-rate limit at 0.1%. The local step gave up progress to keep clear at 34% of contacts, backed away from the route at 3%, and ran from a chasing obstacle at 3%. At these limits (0.5 m/s, 90 °/s), the bottleneck is where the ball can land among the obstacles, not how fast the base can get there.

D. Live Planners Compared

Table 3 compares the live planners on the same 30 roaming courses (6.2 obstacles, 3.2 moving). Only the search changes; sensing, prediction, the arm-angle choice, and the controller are shared. Live hybrid A* reruns the offline space–time search from scratch at each contact against the predictions. The oracle row is the offline planner given the obstacles' true future motion on the same courses.

D* Lite + DWA finished the most courses, fastest, with the fewest collisions; its three failures were one lost dribble and two balls knocked out of bounds after an obstacle hit them. D* Lite alone timed out on 3 courses and lost the ball on 8, 5 of them out of bounds. The price of the local step is path length: rather than standing still while waiting for a gap, it keeps stepping to stay clear of approaching obstacles, so its median path is 52% longer than the straight line. Live hybrid A* needed 5.6 s per replan on average against a 300 ms dribble cycle; the simulation does not charge planning time, so its numbers flatter it, and half its runs timed out. The oracle never collides but plans only 11 of 30 courses: its forward-only motion, time margin, and search budget make it far more conservative than the live planners, so it bounds safety, not completion.

Planner	Done	Coll.	Time	Over.	Replan (ms)
D* Lite + DWA	27/30	13	32.8	52%	119 / 492
D* Lite alone	19/30	19	43.3	35%	85 / 327
Hybrid A* (live)	9/30	29	71.3	19%	5635 / 24419
Oracle	11/30	0	22.3	5%	—

Table 3: Live planners on 30 shared roaming courses. Done: finished while still dribbling. Coll.: distinct collisions summed over all runs. Time: mean seconds over finished runs. Over.: median path length beyond the 6 m straight line. Replan: mean / worst per contact. The oracle found no plan for 19 courses; it never collided on the 11 it planned.

E. Tuning the Live Planner

Each change below was kept only if it held up on the same set of courses.

- **Giving way earlier.** Rejecting velocities that collide within 1.0 s (was 0.6 s) and valuing clearance up to 0.5 m at double weight raised finishes from 25 to 28 of 30 and cut collisions from 47 to 27 on an earlier version of the courses. Making backing up cheaper did no better: stepping aside or waiting is usually the better escape.
- **Speed from clearance.** With the slower dribble, a fixed 0.25 m/s finished 26 of 30 with 25 collisions in 12 runs; speed from clearance finished 25 with 21 collisions in only 7 runs.
- **Running when chased.** On 90 courses, the chase speed-up cost 5 finishes when the top cruise was 0.35 m/s: the robot could not outrun its pursuer. With 0.45 m/s in the open it added 2 finishes, cut collisions by about a quarter, and brought the median run from 37 to 33 s, so we kept both.
- **Predicting bounces.** On 90 courses without random turns, bounce-aware prediction finished 75 against 72 for constant velocity, and adding the longer look-ahead (2 s local, 2.5 s route, 1 s past each landing) reached 76 with 37 collisions instead of 50. With constant-velocity prediction, looking further ahead had made things worse.
- **Uncertainty margins backfired.** Widening each predicted obstacle in proportion to how far ahead it was predicted, every variant finished fewer courses (50–70 of 90 against

72): the wider keep-out zones pushed the robot toward the side lines, where it lost the ball out of bounds. They are off.

- **Shielding the ball** with the chassis from the nearest moving obstacle was mixed (+2 finishes at one speed, −4 at another) and is off.

F. Keeping the Ball in Bounds

Watching runs showed the arm itself sometimes bouncing the ball out: driving along a side line with the ball on the outside, the turn-rate limit kept the arm from swinging it back inward in time, and the fallback for that case ignored the boundary. In a 30-run check, 14 runs aimed at least one bounce outside the court. With in-bounds landing spots and fallback, moves filtered by whether the ball can stay in, the 5 cm margin, and the side-line penalty (Section VI-C), none did. At the same time we began ending each run when the dribble ends: previously the robot was placed beside the ball even after the dribble was lost, so a ball rolling over the finish line counted. On 90 courses, the old count of 84 finishes fell to 77 with the bounds fixes and to 71 under honest counting.

G. Robustness to Unpredictable Obstacles

Table 4 runs D* Lite + DWA on 60 roaming courses with the obstacles' random turns off and on, using bounce-aware or constant-velocity prediction. With turns on, the planner's model of the obstacles is wrong every few seconds.

Turns	Predict	Done	Coll.	OOB	Lost	Time
Off	Bounce	45/60	43	5	8	37.0
On	Bounce	46/60	35	5	8	33.9
Off	Constant	42/60	54	5	11	35.2
On	Constant	45/60	38	7	7	35.9

Table 4: Robustness on 60 roaming courses (6.2 obstacles, 3.1 moving). OOB: ball out of bounds. Lost: dribble lost. The rest of the failures are timeouts.

Random turns do not hurt: completion is within one course with or without them, for either predictor. Bounce-aware prediction finishes as many or more courses with fewer collisions in both settings, most clearly without turns (54 to 43 collisions), where its model is exact. Seeds 30–59 are harder than the 30 shared courses of Table 3: the default finishes 19 of them, against 27 of the first 30. Of its 14 failures, 11 began with an obstacle hitting the ball; the other two balls out of bounds were the arm's, and one run timed out.

H. Energy Consistency

Over 341 live runs, the ledger left on average 0.06% of paddle work unexplained. In the 283 runs where no obstacle struck the ball it closed exactly (to 10^{-14}); the remainder, at most 1.8%, is energy exchanged in ball–obstacle collisions. In a typical finished run the paddle did 143 J of work; the floor took 121 J, impacts 19 J, and the integrator 4 J. Paddle demands stayed modest: at most a 23.9 ° face tilt and 1.18 m/s, and no contact ever hit the tilt limit.

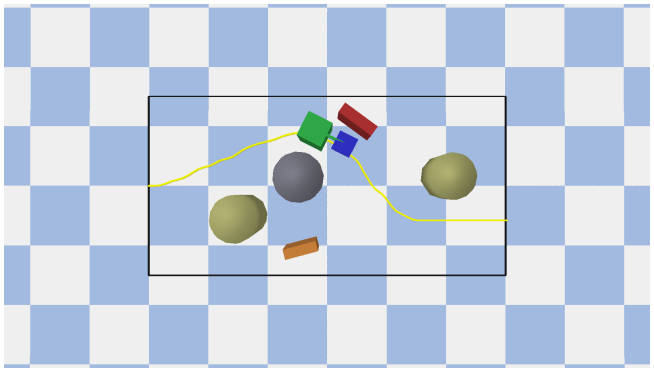


Figure 3: Overhead view of a roaming course with the oracle's planned robot-body path (yellow) and the robot at 45% of the way through.

I. Comparison with an Obstacle-Free Baseline (Objective 5)

TO DO: Run `python experiments.py density`: completion, collisions, path overhead, and redirect frequency vs. obstacle count (0, 2, 4, 6, 8), the 0-obstacle row being the obstacle-free baseline, for D* Lite + DWA and the oracle. Add a figure of completion and collisions vs. density.

J. Terrain Variation

TO DO: Planned work (Rudy): dribbling up and down slopes, then over varied terrain. Describe the approach and results once implemented. On a slope the floor's tilted normal pushes the ball downhill at every bounce and changes bounce timing, so the controller's ballistic model and the paddle aim must account for terrain height and slope.

K. Demonstration

TO DO: Record and link demo videos (`--video`): an offline run with oscillating obstacles and a live run with roaming obstacles.

VIII. Implementation and Running the Code

Code is pure Python (NumPy, PyBullet).

TO DO: Add the repository link and a README with setup steps.

Typical commands:

```
python simulation.py --seed 4
python simulation.py --planner offline
python simulation.py --speed 1 --video r.mp4
python experiments.py planners
python experiments.py redirect
python experiments.py robustness
```

`--planner` selects `dwa` (default), `dstar`, `hybrid`, `offline`, or `oracle` (the offline planner on the live planners' course); `--motion` selects oscillating or roaming obstacles; `--speed` sets GUI playback as a multiple of real time (default 3, 0 for unpaced); `--headless` runs without a window. Solved courses are cached on disk per seed, keyed on the planner, obstacle, and configuration source. `experiments.py` writes raw rows to `results/`.

File	Contents
`simulation.py`	Entry point; dribble loop, controllers, energy ledger, metrics
`planning.py`	Space–time hybrid A*, arm-angle search, no-redirect baseline
`live_planning.py`	Sensor, prediction, D* Lite + DWA, live hybrid A*
`dstar_lite.py`	D* Lite on an 8-connected grid
`obstacles.py`	Course generation, roaming motion, collision tests
`physics.py`	Ballistic timing, energy-consistent paddle impact
`robot.py`	Chassis, arm, paddle
`evaluation.py`	Redirect events, binding-constraint analysis
`experiments.py`	Batch experiments behind every results table
`planner_benchmark.py`	Offline planner benchmark
`render_scene.py`	Report figures
`config.py`	Shared constants and limits

Table 5: Source files.

IX. Responding to Status-Update Feedback

No-redirect baseline (Raghu Vadlamani): using our own planner with redirection disabled as the baseline directly tests the coupling hypothesis (Table 2).

Backtracking when the arm search fails (Imaan Haider): on the original sparse courses no failures were of this kind, but on dense ones 11 of 60 are, so retrying another body path is now worthwhile future work.

What counts as "good enough" (Megan Carlson): Section III, including what counts as a finish.

Is the bottleneck landing spots or base speed? (Thomas DeSilva): we added base speed and turn-rate limits and logged what binds; at these limits it is obstacles (Section VII-C).

Source of failures, and drift from the assumed bounce (Won Jun Lee): failures are broken down by stage in Table 2 and by outcome in Table 4. The drift question pointed at a real problem: the robot fell up to 19 s behind its plan, fixed by tracking the plan's clock (Section VII-A).

Dynamic obstacles such as another person (Sameer Augustine): the live planner (Section VI-C), now with obstacles that turn unpredictably, move at different speeds, and react to the robot.

Energy consistency of the mirror law (Prof. Flickinger): the contact is now a rigid-body impact with full energy accounting (Section V).

X. Adaptability: Challenges and How We Adapted

Reactive → proven-in-advance arm planning (status update). A live arm-angle controller could only check that some angle was clear at a point, never that the arm could get from one safe angle to the next, so we replaced it with the transition-verifying search.

Ball-centric → body-centric planning (status update). The chassis has the larger footprint, so the search plans its center directly and derives the ball's path from it.

A proof is only as good as its timing. Adding time to the search made plans collision-free on paper, but replaying real positions against real obstacle positions found 4 of 5 courses colliding: the old waypoint follower tracked *where* the ball was but never *when*. We moved the controller onto the plan's clock, started the ball where the plan starts it, and added speed limits and a time margin; collisions went to zero. Measuring against ground truth, not the planner's own model, is what exposed the gap.

Known schedule → live replanning. Roaming obstacles break the offline planner's assumption that the future is known. Rather than stretch the offline guarantee, we built a live planner that only senses the present and kept the offline planner as an oracle, accepting that collisions become a measured outcome rather than a guaranteed zero.

Choosing the live algorithm by measurement. D* Lite alone [6] hard-blocked every cell an obstacle might soon reach, which closed off the lane on dense courses and left the robot standing still while obstacles ran into it. Moving the dynamic reasoning into a dynamic-window step [7] worked better; replanning the offline hybrid A* [5] live did worse and was too slow. We kept D* Lite + DWA.

Making the obstacles harder on purpose. Once bounce-aware prediction made obstacles nearly perfectly predictable, a high completion rate no longer showed the planner was robust. We gave obstacles random turns and individual speeds and let them push off the robot, then measured how much the planner degraded (Table 4). Speeds had earlier been capped at the robot's pace because fast obstacles caught the ball from behind; we lifted the cap for variety and answered it with the chase speed-up instead.

Our own claims needed checking. Answering feedback, we found the status update's "6–9 obstacles" partly counted obstacles the generator had placed inside one another when they did not fit; we fixed the generator and relaxed the spacing to reach about 6 real obstacles. Our no-redirect baseline twice solved 0 of 60 courses, a result that looked decisive but was a bug both times: first the body search chose turns a rigidly attached ball could not follow, then a start-line rule rejected the very first pose because part of its arm-angle window sampled the ball a few centimetres behind the line. Fixed, it solves 7. The energy ledger initially left 8% of paddle work unexplained, traced to PyBullet's default drag. Counting a finish whenever the robot crossed the line overstated our results by 6 of 90 courses.

Intuitive fixes that failed. Slowing near moving obstacles and widening predicted obstacles for uncertainty both sound safer, and both made things worse: the first gave obstacles more time to arrive, the second crowded the robot against the side lines. We only learned this by measuring each change against the same courses.

Bugs found by testing against references. D* Lite occasionally returned a slightly longer path; comparing against Dijkstra traced this to floating-point ties between keys end-

ing the search one expansion early. Some courses never finished even with no moving obstacles: a robot at $x = 5.96$ m rounded into a goal cell centered on 6.0 m and stopped 4 cm short.

Simulation speed. The GUI ran at about a tenth of real time. Profiling showed 41 of 46 s spent in PyBullet debug-line calls, each waiting about 20 ms on WSL's software renderer; drawing the arm as a real body and the route as one batched point set brought runs to several times real time.

TO DO: Add any further challenges from the remaining weeks, including terrain.

XI. Limitations and Future Work

The live planner offers no safety guarantee: an obstacle that turns unpredictably can still reach the ball, and nearly every remaining failure starts that way. Recovering a knocked ball, rather than ending the run when the dribble is lost, is the most direct route to more finishes. Prediction assumes the robot knows how obstacles bounce, and the simulation does not charge planning time. The live planner's active avoidance lengthens paths; a smoothness term in the local step did not help, and better waiting behavior is open. The offline body search ignores the arm, which costs 11 of 60 dense courses; backtracking to a different body path when the arm search fails is the natural fix. The offline planner cannot reverse, and its tracking now misses the 5 cm target by 1.3 cm at worst.

TO DO: Stretch goal 2 (multi-bounce lookahead): done or not, and what was learned.

TO DO: Stretch goal 3: physical-feasibility discussion (sensing, paddle actuator bandwidth, ball tracking). The paddle tilt and speed reported in Section VII are a starting point.

XII. Conclusion

TO DO: Write once the final evaluation is in.

Appendix: Individual Contributions

Rudy Arsenault

TO DO: Contributions since the status update (see the status report for earlier work), including terrain variation.

Tracy Yang

TO DO: Contributions since the status update (see the status report for earlier work).

References

[1] M. Bühler, D. E. Koditschek, and P. J. Kindlmann, "Planning and control of robotic juggling and catching tasks," *The International Journal of Robotics Research*, vol. 13, no. 2, pp. 101–118, 1994.

[2] G. Bätz, M. Sobotka, D. Wollherr, and M. Buss, "Robot basketball: Ball dribbling – a modified juggling task," in *Advances in Robotics Research*, T. Kröger and F. M. Wahl, Eds. Berlin, Heidelberg: Springer, 2009, pp. 323–334.

[3] G. Bätz, K.-K. Lee, D. Wollherr, and M. Buss, "Robot basketball: A comparison of ball dribbling with visual and force/torque feedback," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2009.

[4] Y. Ji, G. B. Margolis, and P. Agrawal, "DribbleBot: Dynamic legged manipulation in the wild," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2023.

[5] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *The International Journal of Robotics Research*, vol. 29, no. 5, pp. 485–501, 2010.

[6] S. Koenig and M. Likhachev, "D* Lite," in *Proc. AAAI Conf. on Artificial Intelligence*, 2002, pp. 476–483.

[7] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

[8] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.